

NAVRUNA

Offline Maritime AI Simulation Lab

A Comprehensive Technical Report

PROJECT BASED LEARNING REPORT

Prepared by
NAVRUNA Project Team

October 2026

Abstract

Maritime route planning must account for destination progress, fuel limits, changing ocean conditions, and the risk of land collision. NAVRUNA is an offline-first simulation lab for studying these challenges with reinforcement learning. Unlike a route-following demonstration, the project gives each simulated vessel an origin, a destination, a fuel budget, and a local synthetic ocean state. A shared Proximal Policy Optimization (PPO) policy selects heading changes from observations at each simulation step.

The software combines a FastAPI backend, a PyTorch PPO trainer, a multi-vessel simulator, a bundled port catalog and coastline dataset, and a browser interface built around CesiumJS. Each policy observation contains 25 normalized values describing vessel state, destination geometry, fuel, weather, and nearby land clearance. The policy chooses among seven discrete heading changes. A separate geometry-based safety layer screens movement and checks the swept path against coastline data, so the learned policy does not have final authority over land collision constraints. Procedural weather fields provide spatially and temporally coherent wind, waves, currents, storms, precipitation, visibility, and pressure without requiring a live data feed.

The repository documents an implemented research prototype with training telemetry, fleet analytics, a weather visualizer, and local simulation assets. It does not provide a completed benchmark study establishing navigation success, fuel savings, or real-world safety. Accordingly, this report describes the implemented design and its evaluation approach without presenting unmeasured performance claims. NAVRUNA is intended for software engineering education and simulation research, not for controlling real vessels.

Keywords: maritime simulation, reinforcement learning, PPO, autonomous navigation, collision avoidance, procedural weather

Report Organization

The report introduces the navigation problem and project objectives, reviews relevant reinforcement-learning and geospatial concepts, and then explains NAVRUNA's architecture and methods. Implementation details are followed by an evidence-based analysis of what the repository supports, the limits of current evaluation, and directions for further work.

1 Introduction

1.1 Background

Vessel routing is a sequential decision problem. A vessel must make progress toward a destination while its heading, fuel reserve, nearby coastline, and environmental conditions change over time. A static line between two ports does not represent the repeated decisions needed to navigate under local constraints. Reinforcement learning offers a way to train a policy by interacting with a simulated environment and receiving feedback from its actions.

NAVRUNA turns this idea into an interactive, multi-vessel learning environment. It is designed to run locally with packaged port and land data and deterministic synthetic weather. Users can launch a fleet, advance simulation time, observe telemetry on a globe, and run PPO training while viewing learning diagnostics.

1.2 Problem Statement

Educational demonstrations of autonomous navigation often focus on a learned policy without making environmental assumptions, safety constraints, and model-training behavior visible. Other route visualizations show a precomputed path, which can obscure the decision process. NAVRUNA addresses this gap by providing an inspectable simulation in which a policy selects incremental heading changes, environmental conditions are explicit, and a separate safety shield validates movement against coastline geometry.

1.3 Significance

The project brings together reinforcement learning, geospatial data processing, procedural environmental modeling, API design, and interactive visualization in one software system. Its main contribution is a reproducible classroom and research platform for examining how a navigation policy behaves under synthetic conditions and hard geometric constraints. The distinction between a learned proposal and an authoritative safety check also makes an important systems-engineering principle visible: safety-critical constraints should not depend solely on a policy's learned behavior.

1.4 Report Scope

This report covers the current software visible in the NAVRUNA repository and its accompanying project documentation. It explains the simulator, PPO training path, coastline checks, backend endpoints, and browser interface. It does not claim that the synthetic environment is a validated model of real ocean conditions or that the simulator's safety checks are suitable for operational navigation.

2 Objectives and Scope

2.1 Project Objectives

1. Create an offline-first global maritime simulation using a local port catalog and bundled coastline data.
2. Represent vessel navigation as sequential heading decisions instead of supplying the policy with a precomputed route polyline.
3. Train a shared PPO policy from normalized vessel, destination, fuel, weather, and land-clearance observations.
4. Enforce coastline constraints through a deployment-time action shield that evaluates candidate swept movements.
5. Model wind, waves, currents, storms, precipitation, visibility, and pressure with a deterministic synthetic weather field.
6. Expose simulation, training, and weather information through a FastAPI service and present it in a browser-based globe and training dashboard.
7. Record useful simulation and training telemetry so behavior can be inspected and future experiments can be compared.

2.2 System Scope

Included in the prototype	Outside the validated scope
Synthetic port-to-port voyages and multi-vessel fleet simulation	Live vessel control or operational navigation advice
Bundled port and coastline data; local weather generation	Live AIS, forecast, bathymetry, or nautical-chart integration
PPO training, telemetry, checkpoints, and policy inference	A published benchmark proving generalization or fuel savings
Action masking and a 5 km coastline safety buffer in simulation	COLREGS compliance, certified collision avoidance, or maritime assurance
CesiumJS visualization, scenario controls, and analytics	A fully air-gapped map stack; the globe references CDN assets and an online basemap

2.3 Intended Users

The intended users are students, developers, and researchers exploring reinforcement learning and simulation software. The project supports controlled experimentation with synthetic scenarios and inspection of training behavior. It is not intended to replace a qualified navigator, charting system, or vessel safety system.

3 Review of Related Concepts

3.1 Reinforcement Learning for Sequential Control

In reinforcement learning, an agent observes a state, chooses an action, and receives a reward and a new state from an environment. This interaction is repeated over an episode. For navigation, the agent's state can describe vessel position and heading, destination direction and distance, remaining fuel, weather, and local obstacles. The reward can encourage progress while penalizing resource use and unsafe outcomes. The quality of a learned policy depends on the environment, reward definition, observation design, and range of training scenarios.

3.2 Proximal Policy Optimization

Proximal Policy Optimization is a policy-gradient method that updates a stochastic policy using a clipped objective intended to limit overly large changes between successive policies. The method is commonly paired with a value function and generalized advantage estimation. NAVRUNA implements PPO with a shared neural-network body and separate policy and value outputs. Its trainer collects rollouts from parallel environments, masks actions during training, computes advantages, and performs several optimization epochs per update [1, 2].

3.3 Geospatial Data and Collision Constraints

A geographic safety check must account for the path traveled between two positions, not only the endpoints. NAVRUNA packages coastline polygons and uses both a raster distance field for fast screening and vector geometry for final validation. The global dataset is derived from GSHHG, a hierarchical shoreline and geographic database [3]. This hybrid method supports fast policy observations while retaining a more detailed geometric check along proposed movement segments.

3.4 Interactive Geospatial Visualization

CesiumJS provides a browser library for interactive 3D globes and maps [4]. In NAVRUNA, the globe displays vessel positions and status; separate panels expose fleet measures, vessel telemetry, weather layers, and training controls. This presentation connects policy decisions to their spatial context and makes the simulation easier to inspect than a terminal-only workflow.

3.5 Design Implications

Three design choices follow from these concepts. First, the policy receives local state rather than a prescribed route. Second, the simulated world provides repeatable, synthetic inputs for controlled learning experiments. Third, collision screening remains a distinct software layer, so the learned policy may propose an unsafe action without that action being executed in the deployment simulation. These choices improve experiment visibility but do not, by themselves, demonstrate real-world validity.

4 System Architecture

NAVRUNA is organized as a local web application with a simulation API, simulation and training models, bundled geospatial data, and a browser interface. The backend initializes computationally heavy components lazily so the health endpoint can respond before the coastline grid and PyTorch model are loaded.

Layer	Implementation	Responsibility
User interface	HTML, CSS, JavaScript, CesiumJS	Fleet controls, globe, telemetry, weather layers, analytics, training charts
HTTP API	FastAPI	Simulation lifecycle, fleet state, analytics, ports, weather, policy and training routes
Fleet simulator	Python	Creates vessels, advances voyages, applies decisions, updates metrics and events
Navigation policy	PyTorch PPO	Maps 25-value observations to seven heading-change scores; estimates state value
Safety and geography	Shapely, NumPy, SciPy	Loads land polygons, builds distance grid, screens actions, verifies swept segments
Weather model	Deterministic procedural Python model	Produces coherent ocean conditions at a location and simulation time
Local data	JSON and GeoJSON assets	Port locations, coastline polygons and frontend copies of simulation data

4.1 Request and Simulation Flow

1. The browser sends a request to start a scenario or advance the simulator through the `/api/simulation` routes.
2. The fleet simulator obtains each active vessel's observation and asks the policy for candidate heading scores. Before movement, it validates actions against the coastline safety geometry.
3. The environment updates vessel position, heading, fuel, progress, weather values, and episode outcomes. Completed voyages are assigned a new destination; failures and collision events are recorded.
4. The API returns fleet state and analytics to the browser, which refreshes the globe and telemetry panels. The training page separately polls training progress and history.

4.2 Principal API Routes

Route	Purpose
GET <code>/api/health</code>	Liveness and offline-mode status
POST <code>/api/simulation/start</code>	Generate a fleet with a selected vessel count and optional seed
POST <code>/api/simulation/step</code>	Advance simulated time
GET <code>/api/simulation/state</code>	Return the current vessel snapshot and metrics
GET <code>/api/simulation/analytics</code>	Return fleet and trainer analytics
GET <code>/api/simulation/weather</code>	Return a procedural global weather field
POST <code>/api/simulation/train</code>	Start PPO training in a background worker
GET <code>/api/simulation/training</code>	Return current training status, report, history, and logs

5 Research Methodology

5.1 Research Design

The project follows an experimental software-engineering design. The environment, learning algorithm, safety system, API, and visualization are separate enough to inspect individually while remaining integrated in one simulation workflow. A scenario seed can be supplied when creating a fleet, which supports repeatable simulation starts. Training environments use configured seeds and a staged curriculum.

5.2 Environment State and Action Space

Each vessel is modeled as an episode with an origin and destination selected from the local port catalog. The environment positions each endpoint offshore and initializes heading, speed, and fuel. The policy receives a 25-dimensional normalized observation. It includes position and heading, relative destination bearing and distance, fuel fraction, speed, progress, local wind and current direction, wave height, storm intensity, seven directional land-clearance probes, and normalized step count.

Element	Project configuration
Observation	25 normalized values for vessel, target, fuel, environment, and land proximity
Action	Seven discrete heading changes: -35° , -20° , -10° , 0° , $+10^\circ$, $+20^\circ$, $+35^\circ$
Episode success	Destination reached within 20 nautical miles
Terminal failure	Land collision, fuel exhaustion, or configured time/step limit
Terminal rewards	+2 for destination reached; -2 for collision or fuel/time failure
Intermediate shaping	Small reward for distance progress, with fuel-use and low-clearance penalties

5.3 Training Procedure

The PPO trainer creates parallel environments, collects rollout observations and rewards, and uses a shared policy/value network with 256-, 256-, and 128-unit hidden layers. The policy has seven output logits and the value head has one output. Training uses a learning rate default of 0.0003, discount factor 0.99, GAE lambda 0.95, a clipping range of 0.2, entropy coefficient of 0.01, and gradient clipping. The curriculum begins with shorter routes and gradually increases route length; land constraints are enabled after the early open-water phase. Training status and update metrics are written to local history and checkpoint files.

5.4 Safety Method

The policy's action is checked before execution. A fast raster-based mask screens likely unsafe directions, then vector coastline geometry checks the swept movement segment against a configurable 5 km buffer. The fleet simulator uses the verified candidates and does not execute a movement when no safe candidate remains. During training, collision can also terminate an episode with a penalty; masking is used to reduce unsafe samples. The safety guarantee applies only to this simulation and the resolution and accuracy of its bundled data.

5.5 Weather and Data

The weather simulator deterministically calculates conditions from latitude, longitude, time, and a seed. It supplies wind speed and direction, wave height, current speed and direction, storm intensity, precipitation, visibility, pressure, and sea state. The bundled port catalog contains 279 locations. Coastline polygons are stored in a local GeoJSON asset, and the backend builds a 0.25-degree land and distance grid for fast queries. The frontend also contains local copies of port and land assets.

6 Implementation Details

6.1 Backend and Simulation API

The FastAPI application serves API routes and the static frontend. The health endpoint reports the offline synthetic mode without importing heavy model dependencies. Simulation, weather, and trainer objects are initialized on demand. Fleet stepping is moved to worker threads so synchronous simulation work does not block the async request loop. Training is submitted to a single-worker executor, and a lock reserves the trainer's starting state to prevent duplicate concurrent submissions.

6.2 Geospatial Safety

The geography module loads polygon features, builds their union, and prepares the geometry for repeated spatial queries. A raster grid supports quick land lookup and approximate distance-to-land queries. For movement validation, the simulator samples points along the proposed segment and checks both direct polygon intersection and proximity to the safety buffer. This continuous segment check is designed to prevent a vessel from skipping across a narrow land feature between safe endpoints.

6.3 Fleet and Voyage Management

A scenario can contain between 1 and 300 vessels through the API request model; the interface defaults to 60. A simulation step advances the fleet in hourly increments and supports fractional time steps. Each vessel maintains navigation state, fuel use, weather telemetry, risk indicators, progress, and its most recent action. A successful voyage increments completion metrics and assigns a new port pair. A collision, no-safe-action result, fuel exhaustion, or time limit is recorded as a failure event.

6.4 Browser Interface

The main screen combines a 3D globe with mission controls, a selectable vessel inspector, fleet outcomes, and environment telemetry. The weather visualizer can show waves, wind, currents, or storms. A separate Training Lab presents learning status, episode outcomes, training speed, losses, entropy, approximate KL, learning-rate information, and activity logs. The UI also reports model device information and configuration.

6.5 Running the Prototype

On Windows, the project provides `run_navruna.bat` and `start.ps1` launch paths. The launcher starts the API and frontend locally and opens the browser interface. The project README documents the usual local URLs and dependencies. The user can start a simulation, alter fleet size and simulated time, inspect vessel state, open weather layers, and begin or stop training through the interface.

7 Data Analysis and Interpretation

7.1 What Can Be Established from the Repository

The implementation establishes the following design properties: vessel actions are selected step by step; the policy receives a fixed-size observation and discrete action set; the trainer supports parallel PPO rollouts and curriculum progression; the simulation checks candidate paths against coastline geometry; and the browser exposes fleet, weather, and training telemetry. The automated test source includes checks for the port catalog, land geometry, observation and network dimensions, no-preplanned-route fleet behavior, fractional time stepping, voyage reassignment, and no-safe-action failure handling.

7.2 Evaluation Measures

A full empirical evaluation should record voyage success rate, collision count, no-safe-action count, fuel use, distance traveled, route completion time, and policy reward across repeated seeds. Training evaluation should use held-out origin-destination pairs and report uncertainty across multiple training runs. Baselines should include direct-to-destination steering with the same safety shield and a classical path-planning method. Stress scenarios should vary route length, coastline complexity, weather severity, and fuel budget.

Measure	What it would show	Evidence needed
Voyage success rate	Ability to reach destinations under defined conditions	Completed and failed episodes over repeated held-out scenarios
Collision and safety events	Whether the shield prevents unsafe executed movement	Collision counts and records of rejected actions by scenario
Fuel and time use	Resource efficiency compared with a baseline	Consistent fuel model, elapsed simulated time, and baseline runs
Training stability	Learning progress and sensitivity to random seeds	Learning curves, checkpoint metadata, and repeated runs
Runtime performance	Fleet and API responsiveness as scale increases	Timed runs with documented hardware, fleet size, and step size

7.3 Interpretation and Evidence Limits

The current repository provides an implemented simulator and a test suite definition, but no supplied, reproducible benchmark dataset or report of completed PPO evaluation runs. The training dashboard is an observability feature, not a performance result. Therefore, this report does not assign a success percentage, claim fuel savings, or compare NAVRUNA numerically with another navigator. Results should be added only after a controlled experiment records configuration, hardware, seeds, training checkpoint, evaluation cases, and raw outcomes.

8 Limitations and Future Work

8.1 Current Limitations

- Weather is procedural and synthetic. It is suitable for repeatable experiments but does not reproduce forecast uncertainty or observed ocean dynamics.
- The bundled coastline data and raster safety field have finite resolution. Geographic error, projection effects, and omitted bathymetry can affect apparent clearance.
- The simulator uses simplified vessel speed, fuel, and environmental-drift models. These do not capture vessel-specific propulsion, loading, maneuverability, or sea-state response.
- The project does not implement COLREGS-aware interactions, traffic separation schemes, port approach procedures, multi-vessel collision avoidance, or human-supervised operational workflows.
- The main UI currently references external CesiumJS assets and an online basemap, so simulation data are local but the complete visual stack is not fully air-gapped.
- No supplied results establish that a trained policy generalizes to unseen routes, weather conditions, or geographic regions.

8.2 Future Work

1. Build a repeatable evaluation suite with held-out routes, multiple random seeds, explicit baseline policies, and confidence intervals.
2. Add bathymetry, vessel draft, lane constraints, port arrival windows, and vessel-specific dynamics to the environment.
3. Replace or supplement procedural weather with replayable historical datasets while preserving deterministic experiment modes.
4. Model COLREGS rules, traffic interactions, and uncertainty in sensor and geographic information before studying operationally relevant scenarios.
5. Track experiments and checkpoints with configuration hashes so results can be reproduced and compared.
6. Self-host the CesiumJS distribution and imagery assets for installations that need a fully air-gapped interface.

8.3 Research Boundary

NAVRUNA is a synthetic simulation and software-engineering project. Its coastline shield is a prototype software constraint, not a certified navigation safety system. Real-world use would require validated charts and bathymetry, vessel dynamics, COLREGS-aware behavior, uncertainty modeling, independent verification and validation, and qualified human oversight.

9 Conclusion

NAVRUNA brings together a local maritime simulation, a PPO learning system, a geometry-based coastline shield, procedural weather, a FastAPI backend, and a browser-based geospatial interface. Its stepwise action model makes the policy's navigation task explicit, while training telemetry and vessel inspection tools help users understand the simulation's behavior. The architecture is suitable for classroom demonstrations and controlled research into reinforcement-learning navigation.

The current evidence supports a description of what the software implements, not a claim that the learned policy is already effective or operationally safe. A rigorous next step is to evaluate repeatable held-out scenarios against transparent baselines and report navigation, safety, fuel, time, and runtime metrics. Keeping those conclusions tied to measured results will strengthen NAVRUNA as a research prototype and provide a clear path for future improvements.

References

- [1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347. <https://arxiv.org/abs/1707.06347>
- [2] Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438. <https://arxiv.org/abs/1506.02438>
- [3] Wessel, P. and Smith, W. H. F. (1996). A Global, Self-consistent, Hierarchical, High-resolution Shoreline Database. *Journal of Geophysical Research: Solid Earth*, 101(B4), 8741-8743. <https://doi.org/10.1029/96JB00104>
- [4] Cesium. CesiumJS Fundamentals. Official learning documentation. <https://cesium.com/learn/cesiumjs-fundamentals/>
- [5] FastAPI. FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [6] NAVRUNA project repository. README, architecture notes, PPO model card, source code, and automated test definitions. Project files supplied with this report, accessed October 2026.